

Problems on TREES

DAY 174
31/12/2025

Problem 5 Give an algorithm for Inserting an Element into Binary Tree

Solution: Since the given tree is a Binary Tree, we can insert the element whenever we want. To insert an element, we can use the level order traversal and insert the element whenever we find the node whose left or right child is Null.

Algorithm

1. If $root == Null$
 - Create a new node with data
 - Make it the root
 - Return root
2. Create an empty queue Q
3. Enqueue root into Q
4. While Q is not Empty:
 1. Dequeue a node temp from Q
 2. If $temp.left == Null$
 - Insert new node as temp.left
 - Break
 3. Else Enqueue temp.left
 4. If $temp.right == Null$
 - Insert new node as temp.right
 - Break
 5. Else Enqueue temp.right
5. END

code

```
public class BinaryTreeInsertion {  
  
    // Method to insert an element into a Binary Tree  
    public static TreeNode insert(TreeNode root, int data) {  
  
        // Case 1: Empty tree  
        if (root == null) {  
            return new TreeNode(data);  
        }  
  
        Queue<TreeNode> q = new LinkedList<>();  
        q.add(root);  
  
        // Level order traversal  
        while (!q.isEmpty()) {  
  
            TreeNode current = q.poll();  
  
            // Insert as left child if empty  
            if (current.left == null) {  
                current.left = new TreeNode(data);  
                break;  
            } else {  
                q.add(current.left);  
            }  
  
            // Insert as right child if empty  
            if (current.right == null) {  
                current.right = new TreeNode(data);  
                break;  
            } else {  
                q.add(current.right);  
            }  
        }  
  
        return root;  
    }  
}
```

Problem 6 Give an algorithm for finding the Size of Binary Tree

Solution

Calculate the Size of left & Right Subtrees Recursively, add 1 (Current node) and return to its parent.

Algorithm : Size BT (root)

1. If $root == NULL$
→ Return 0
2. Recursively find Size of left Subtree
3. Recursively find Size of Right Subtree
4. Return
 $Size = leftSize + rightSize + 1$

code

```
public static int size(TreeNode root) {  
  
    // Base case  
    if (root == null) {  
        return 0;  
    }  
  
    // Size = left subtree + right subtree + root  
    return size(root.left) + size(root.right) + 1;  
}
```

Problem 7 Write an Algorithm for finding the Size of Binary Tree Without Recursion.

Solution $\Rightarrow$ Yes, using level order traversal

Algorithm

1. if root == NULL
Return 0
2. Initialize Count = 0
3. Create an Empty Queue Q
4. Enqueue root
5. While Q is not Empty
 1. Dequeue a node temp
 2. Increment Count
 3. if temp.left != null
Enqueue it
 4. if temp.Right != null
Enqueue it
6. Return Count

Code

```
public class BinaryTreeSizeIterative {  
    public static int size(TreeNode root) {  
        if (root == null) return 0;  
  
        int count = 0;  
        Queue<TreeNode> q = new LinkedList<>();  
        q.add(root);  
  
        while (!q.isEmpty()) {  
            TreeNode current = q.poll();  
            count++;  
  
            if (current.left != null) q.add(current.left);  
            if (current.right != null) q.add(current.right);  
        }  
  
        return count;  
    }  
}
```